SEC101: SCIENTIFIC WRITING AND COMPUTATIONAL ANALYSIS TOOLS

M.Sc. Computer Science

Submitted By:

Harshit Kumar More
Roll No: 16
Department of Computer Science

Submitted To:

Bhomik Kaushik

November 22, 2025

Problem Statement: 01

Problem: You are given the matrix and vector.Perform following Operations:

1) Display the matrix A and vector b using MATLAB

2) Compute the determinant of Matrix A

3) Find the Inverse of Matrix A

4) Compute the Eigen values of Matrix A

5) Create new Matrix M=reshape(1:12,3,4) and perform the transpose and Horizontally concatenate M with a column of ones to form C=[M , 1] and display the result

Below is the MATLAB script:

```
% matrix_array_ops.m
% Matrix creation, indexing, solving linear systems, eigenvalues.

clc; clear; close all;
A = [4 1 2; 0 3 -1; 1 0 2];
b = [7; 4; 3];

% Display
disp('Matrix A:'); disp(A);
disp('Vector b:'); disp(b);

% Matrix operations
detA = det(A);
invA = inv(A);              % small matrix ok
x = A\b;                    % solve Ax = b (preferred)
eigVals = eig(A);

fprintf('det(A)=%g\n', detA);
disp('Inverse of A:'); disp(invA);
disp('Solution x = A\\b:'); disp(x);
disp('Eigenvalues:'); disp(eigVals);

% Reshape and concatenate
M = reshape(1:12, 3, 4); % 3x4 matrix
```

```
25 M_t = M';                    % transpose
26 C = [M, ones(3,1)];          % horizontal concat
27
28 disp('Example reshape M:'); disp(M);
29 disp("Transpose:"); disp(M_t)
30 disp("Horizontal Concat:"); disp(C)
```

Output:

0.50

```
Matrix A:
     4     1     2
     0     3    -1
     1     0     2

Vector b:
     7
     4
     3

det(A)=17
Inverse of A:
    0.3529   -0.1176   -0.4118
   -0.0588    0.3529    0.2353
   -0.1765    0.0588    0.7059

Solution x = A\\b:
    0.7647
    1.7059
    1.1176

Eigenvalues:
    1.1206
    4.5321
    3.3473
```

0.50

```
Example reshape M:
     1     4     7    10
     2     5     8    11
     3     6     9    12

Transpose:
     1     2     3
     4     5     6
     7     8     9
    10    11    12

Horizontal Concat:
     1     4     7    10     1
     2     5     8    11     1
     3     6     9    12     1
```

Figure 1: Output of Problem 1

Problem Statement: 02

Problem: Write a MATLAB script that demonstrates the basic concepts of scientific data visualization. Your task is to:

1. Generate a vector x over the interval $[0, 2\pi]$ and compute the functions:
$$y_1 = \sin(x), \qquad y_2 = \cos(2x)e^{-0.3x}.$$
2. Create a figure window containing two subplots:
   - **Subplot 1:** Plot $y_1 = \sin(x)$ using a line plot. Include a suitable title, axis labels, and enable the grid.
   - **Subplot 2:** Create a scatter plot of y_2, where the color of each point represents its value. Add title, labels, grid lines, and a colorbar.
3. Save the final plotted figure as a PNG file named `plot_examples.png`.

Below is the MATLAB script:

```
% plotting_visualization.m
% Demonstrates line plot, scatter, subplot, labels and saving figure.

clc; clear; close all;
x = linspace(0,2*pi,200);
y1 = sin(x);
y2 = cos(2*x).*exp(-0.3*x);

figure('Name','Plotting Examples','NumberTitle','off');

subplot(2,1,1)
plot(x,y1,'LineWidth',1.5)
title('y = sin(x)')
xlabel('x'), ylabel('sin(x)')
grid on

subplot(2,1,2)
scatter(x, y2, 8, y2)    % color by value
title('y = cos(2x)*e^{-0.3x}')
xlabel('x'), ylabel('y2')
colorbar
```

```
grid on

% Save as PNG
saveas(gcf, 'plot_examples.png');
fprintf('Saved figure to plot_examples.png\n');

```

Output:

0.50

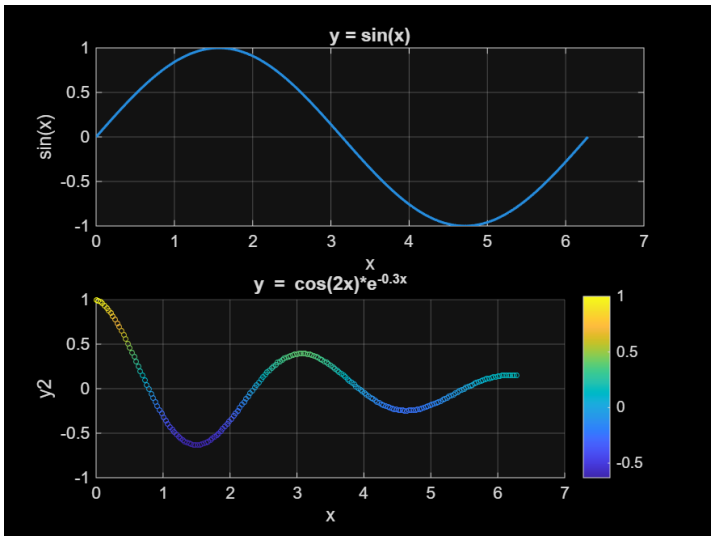


Figure 2: Output of Problem 2

Problem Statement: 03

Problem: Write a MATLAB script that demonstrates fundamental image processing operations using an input image of your choice. Your task is to:

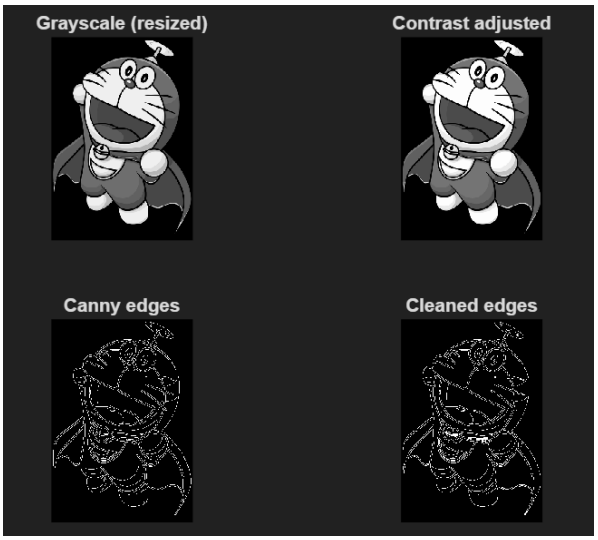
1. Load an input image and display it as the **original image**.
2. Convert the image to **grayscale** only if it contains three color channels (i.e., RGB format).
3. Resize the grayscale image to **60% of the original size** for demonstration purposes.
4. Perform **contrast enhancement** using an appropriate intensity adjustment method.
5. Apply **Canny edge detection** to extract major edges from the enhanced image.
6. Use morphological operations:
   - **Dilation** to strengthen and connect edges,
   - **Erosion** to thin the edges back to a regular thickness.
7. Display the following results in a 2×2 subplot layout:
   - Grayscale (resized) image,
   - Contrast-adjusted image,
   - Canny edge map,
   - Morphologically cleaned edge image.

Below is the MATLAB script:

```
% image_processing_demo.m
% Use the uploaded screenshot, convert to grayscale, enhance and detect edges.

clc; clear; close all;
%clc → clears the Command Window
%clear → removes variables from workspace
%close all → closes all open figure windows


```

```
I = imread("doremon.png"); %Stores it as matrix I

figure('Name','Original Image'), imshow(I), title('Original');

% Convert to grayscale if needed
if size(I,3) == 3  % If the image has 3 color channels → it is RGB → convert to graysc
    Ig = rgb2gray(I);
else
    Ig = I;
end

% Resize for demonstration
Ig_small = imresize(Ig, 0.6); %Shrinks image to 60% of original size

% Contrast enhancement
Ig_adj = imadjust(Ig_small); % imadjust stretches pixel intensity

% Edge detection
edges = edge(Ig_adj, 'Canny'); %Finds sharp intensity changes

% Morphological cleanup  %Edges might be thin, broken, or noisy.
edges_clean = imdilate(edges, strel('disk',1)); %Dilation: thickens edges & connects b
edges_clean = imerode(edges_clean, strel('disk',1)); %Erosion: thins edges back to nor

% Display
figure('Name','Image Processing Results');
subplot(2,2,1), imshow(Ig_small), title('Grayscale (resized)');
subplot(2,2,2), imshow(Ig_adj), title('Contrast adjusted');
subplot(2,2,3), imshow(edges), title('Canny edges');
subplot(2,2,4), imshow(edges_clean), title('Cleaned edges');

% Save results
imwrite(Ig_adj, 'image_gray_adj.png');
imwrite(edges_clean, 'image_edges.png');
fprintf('Saved image_gray_adj.png and image_edges.png\n');
```

Output:

0.50 

0.50 

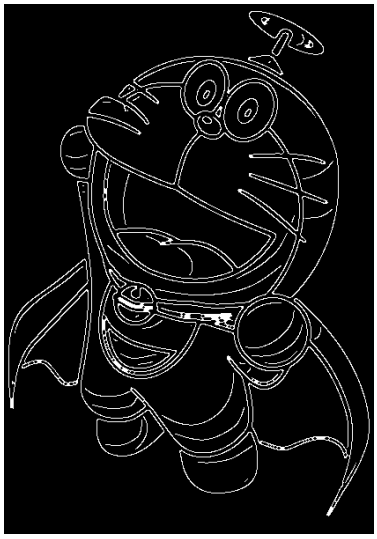
0.50 

Figure 3: Output of Problem 3

Problem Statement: 04

Problem: Write a MATLAB script that converts a given input image into an emoji-based mosaic representation. Your task is to:

1. Read an input image and convert it to **grayscale**. Resize the image to a fixed size (e.g., 300×300) to standardize the processing.
2. Divide the grayscale image into non-overlapping blocks of equal size (e.g., 20×20 pixels).
3. For each block:
   - Compute the **average brightness** value (0–255),
   - Map this brightness level to a corresponding **emoji** selected from a predefined list of colored emojis, where darker emojis represent lower brightness and lighter emojis represent higher brightness.
4. Construct an **emoji-art matrix** in which each block of the grayscale image is replaced by its mapped emoji.
5. Convert the emoji matrix into a properly formatted **emoji text string** suitable for display.
6. Display the resulting emoji-based image representation inside a MATLAB **UI figure** using a `uitextarea` with an emoji-supported font.

Below is the MATLAB script:

```
clc; clear; close all;

img = imread("doremon.png");       % change file if needed
img_gray = rgb2gray(img);

img_gray = imresize(img_gray, [300 300]);


% Emoji mapping based on brightness
emoji_list = ["","","","","","","","",""];
num_emojis = length(emoji_list);

```

```
% Each emoji covers equal brightness range
range_step = 256 / num_emojis;


% Define block size
block_size = 20;      % each emoji = block average
rows = size(img_gray, 1);
cols = size(img_gray, 2);

rows_b = rows / block_size;
cols_b = cols / block_size;

emoji_art = strings(rows_b, cols_b);


% Build emoji matrix
for r = 1:rows_b
    for c = 1:cols_b

        % extract block
        block = img_gray((r-1)*block_size+1 : r*block_size, ...
                         (c-1)*block_size+1 : c*block_size);

        % compute average brightness 0{255
        avg_brightness = mean(block(:));

        % convert brightness → emoji index
        idx = ceil(avg_brightness / range_step);
        idx = max(1, min(num_emojis, idx));

        emoji_art(r, c) = emoji_list(idx);
    end
end


emoji_str = strjoin(join(emoji_art, ""), newline);


fig = uifigure('Name','Emoji Image Compressor', 'Position',[100 100 800 600]);
txt = uitextarea(fig, ...
    'Value', emoji_str, ...
    'FontName','Segoe UI Emoji', ...
    'FontSize', 14, ...
```

```
    'Position',[20 20 760 560]);

% Done!
disp("Emoji compression completed!");
```

Output:

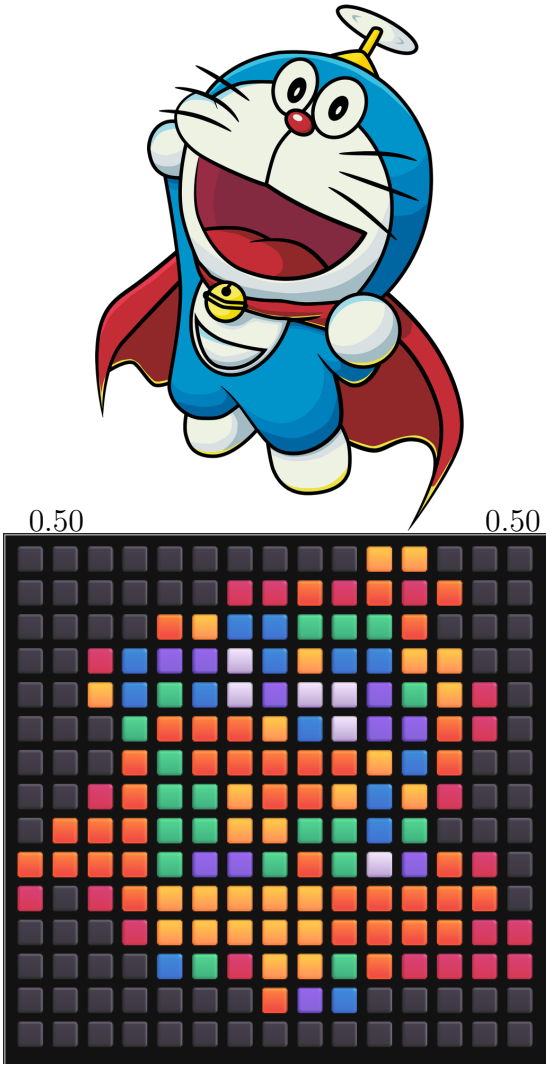


Figure 4: Output of Problem 4